

## Unidad 1: Arquitectura y Diseño.

**Tema 1. Arquitectura. Diseño arquitectónico. Decisiones de diseño arquitectónico. Organización del sistema. Estilos de descomposición modular. Estilos de control. Arquitecturas de referencia. Arquitecturas de sistemas distribuidos. Arquitecturas de aplicaciones**

### Conceptos Fundamentales y Requerimientos

La arquitectura de un sistema es el diseño de alto nivel que afecta directamente su **rendimiento, solidez, grado de distribución y mantenibilidad**. El diseño arquitectónico implica tomar decisiones fundamentales sobre cómo estructurar el sistema y descomponerlo en módulos.

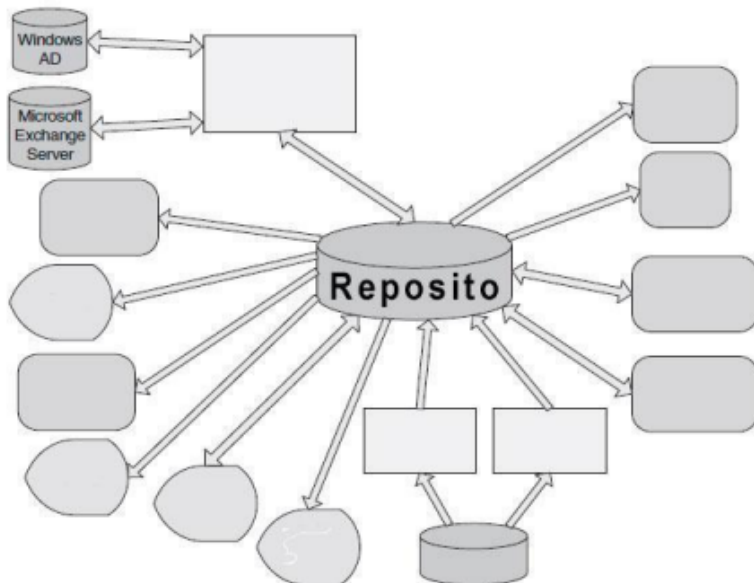
La elección de un estilo arquitectónico depende fuertemente de los **requerimientos no funcionales**:

- **Rendimiento:** Se deben identificar operaciones críticas en pocos subsistemas para minimizar la comunicación entre ellos.
- **Protección:** Se recomienda una **arquitectura en capas**, protegiendo los recursos críticos en las capas más internas.
- **Seguridad:** Las operaciones de seguridad deben localizarse en uno o pocos subsistemas.
- **Disponibilidad:** Requiere incluir componentes redundantes que permitan actualizaciones sin detener el sistema.
- **Mantenibilidad:** El sistema debe diseñarse con componentes independientes de "grano fino" que sean fáciles de modificar.

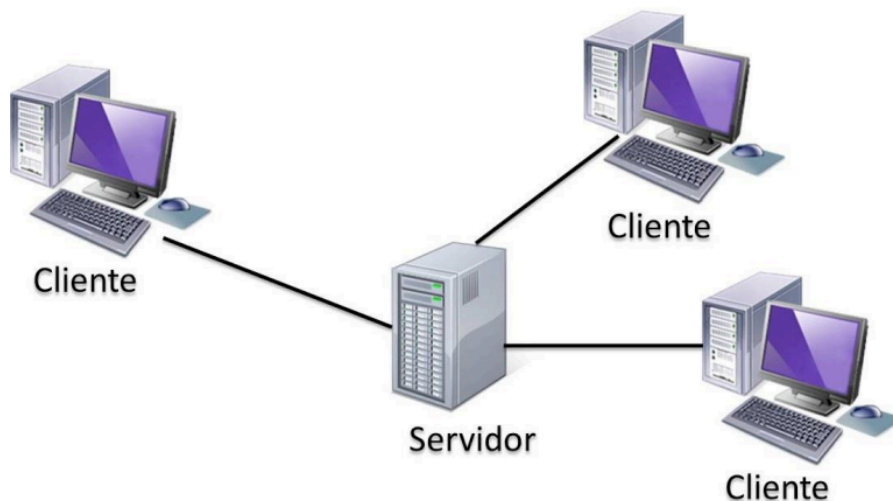
### Organización del Sistema (Modelos Estructurales)

Existen tres estilos organizacionales predominantes para estructurar un sistema completo:

- **Modelo de Repositorio:** Todos los datos compartidos residen en una **base de datos central**. Es eficiente para compartir grandes volúmenes de datos, pero el repositorio puede ser un cuello de botella y dificulta la evolución si los subsistemas tienen modelos de datos distintos.



- **Modelo Cliente-Servidor:** El sistema se organiza en servicios (servidores) y clientes que los usan. Es ideal para **sistemas distribuidos** en red y permite añadir o actualizar servidores de forma transparente



- **Modelo de Capas (Máquina Abstracta):** Organiza el sistema en niveles funcionales, donde cada capa ofrece servicios a la superior. Facilita la

independencia, aunque estructurar sistemas complejos de esta forma puede ser difícil.

|                               |
|-------------------------------|
| Capa de presentación          |
| Capa de lógica de negocios    |
| Capa de persistencia de datos |
| Capa de base de datos.        |

## Descomposición Modular y Control

Un **subsistema** es independiente y tiene interfaces definidas, mientras que un **módulo** suele ser un componente dentro de un subsistema que no funciona por sí solo.

- **Estrategias de descomposición:** Puede ser **orientada a objetos** (objetos que se comunican) u **orientada a flujos de funciones** (módulos que transforman datos).
- **Estilos de control:**
  - **Centralizado:** Un subsistema tiene la responsabilidad total (ej. modelo de **llamada-retorno** en sistemas secuenciales o modelo de **gestor** en concurrentes).
  - **Basado en eventos:** Los subsistemas responden a eventos externos (ej. modelos de **transmisión/broadcast** o dirigidos por **interrupciones** en tiempo real).

## Arquitecturas de Aplicaciones Genéricas

La mayoría de los sistemas pertenecen o combinan una de estas cuatro clases:

1. **Procesamiento de Datos:** Sistemas por lotes (batch) con estructura entrada-proceso-salida. Operan sobre grandes cantidades de datos (nóminas, facturación).
2. **Procesamiento de Transacciones:** Sistemas interactivos centrados en bases de datos. Las transacciones se procesan de forma atómica (ej. sistemas de gestión de recursos).
3. **Procesamiento de Eventos:** El sistema responde a estímulos del entorno o de la interfaz de usuario de forma impredecible (ej. editores de texto, juegos o sistemas de tiempo real).
4. **Procesamiento de Lenguajes:** Traducen un lenguaje formal (natural o artificial) a otro formato para su interpretación (ej. compiladores).

## Arquitecturas de Sistemas Distribuidos

Se enfocan en cómo los procesos interactúan a través de una red:

- **Evolución Cliente-Servidor:** Puede variar desde un solo servidor hasta modelos de **n-niveles** (presentación, procesamiento y datos) o el uso de **servidores Proxy** para caché de recursos.
- **Modelo Igual a Igual (P2P):** Todos los procesos tienen roles similares. Mejora la tolerancia a fallos y escalabilidad, aunque es más difícil de coordinar.
- **Interacción:** Puede ser **Síncrona** (tiempos limitados y conocidos, como en sistemas de control SCADA) o **Asíncrona** (sin limitaciones de velocidad o retardo, como en Internet).

### 1. Arquitectura en Capas (Layers)

- **Concepto:** Divide el software en unidades llamadas capas, donde cada una es una agrupación de módulos que ofrece servicios cohesivos.
- **Regla Fundamental:** Las relaciones de uso deben ser **unidireccionales** y no circulares.

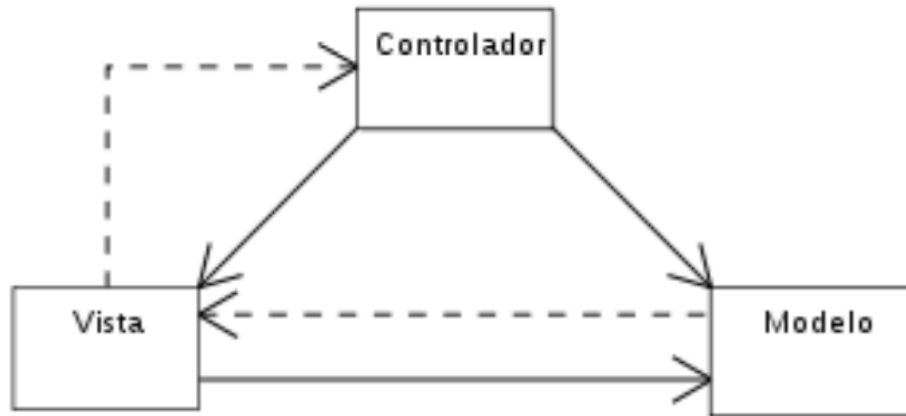
- **Capas comunes:** Presentación, Lógica de Negocio, Persistencia y Base de Datos.
- **Beneficio:** Permite que los módulos se desarrollen y mantengan de forma independiente, soportando portabilidad y reutilización.

## 2. Cliente - Servidor

- **Concepto:** Los clientes solicitan servicios a uno o más servidores que controlan el acceso a recursos compartidos.
- **Componentes:** El **Cliente** (invoca servicios) y el **Servidor** (proporciona servicios). Se comunican mediante un protocolo de **solicitud/respuesta** (ej. HTTP).
- **Debilidades:** El servidor puede ser un **punto único de falla** o un cuello de botella en el rendimiento.

## 3. Modelo - Vista - Controlador (MVC)

- **Concepto:** Separa la funcionalidad de la interfaz de usuario del resto del sistema.
- **Componentes:**
  - **Modelo:** Contiene los datos y la lógica de la aplicación.
  - **Vista:** Produce la representación de los datos para el usuario.
  - **Controlador:** Media entre el modelo y la vista, traduciendo acciones del usuario en cambios.
- **Debilidad:** Su complejidad puede no justificarse para interfaces muy simples.

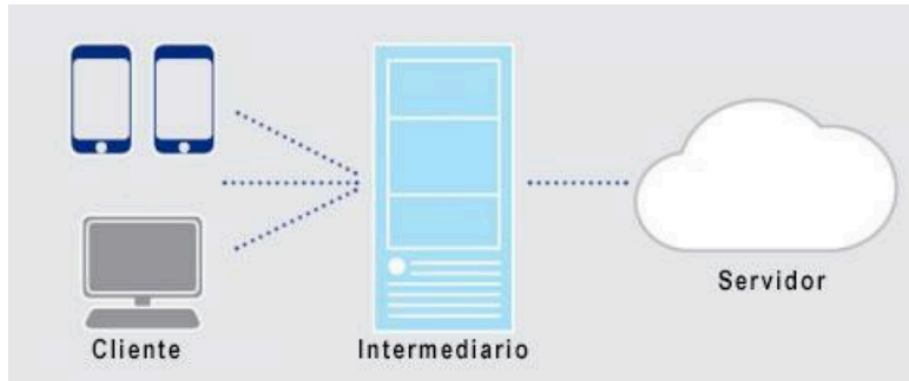


#### 4. Repositorio (Shared Data)

- **Concepto:** Varios componentes independientes acceden y manipulan grandes cantidades de datos compartidos en un almacén central.
- **Interacción:** Los "accesores" realizan operaciones y escriben resultados en el almacén, el cual puede ser iniciado por los accesores o por el almacén mismo.
- **Debilidades:** Riesgo de acoplamiento estrecho entre productores y consumidores de datos, y el almacén central es un posible punto de falla.

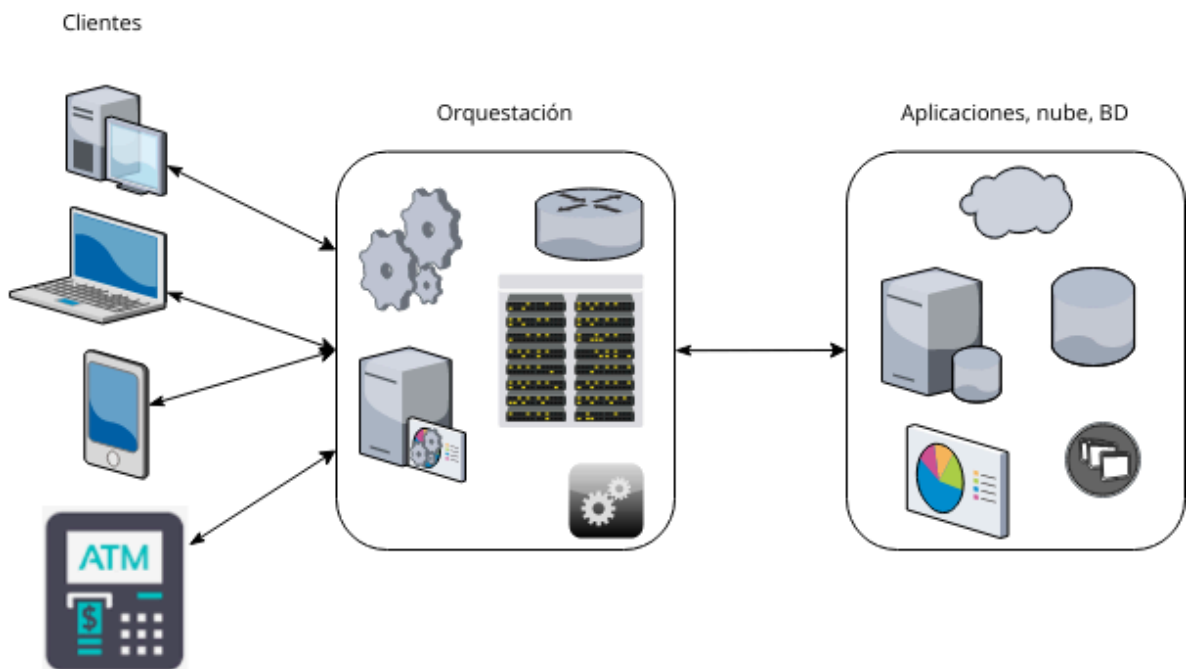
#### 5. Intermediario (Broker)

- **Concepto:** Utilizado en sistemas distribuidos para **desacoplar** clientes de servicios. El Broker coordina la comunicación, reenvía solicitudes y maneja excepciones.
- **Funcionamiento:** Los servicios se registran en el Broker; el cliente solicita un servicio al Broker, quien localiza al servidor adecuado y devuelve el resultado.
- **Ventajas:** Independencia de las partes y flexibilidad para añadir o mover servicios sin afectar al cliente.
- **Desventajas:** Mayor complejidad, respuestas más lentas por la "escala" en el intermediario y riesgo de caída total si el Broker falla.



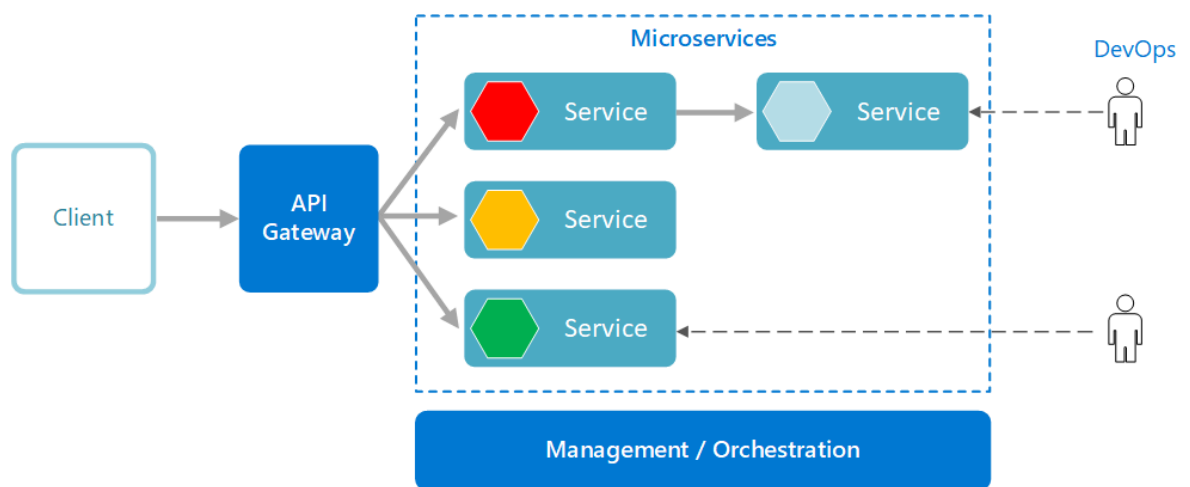
## 6. SOA (Arquitectura Orientada a Servicios)

- **Concepto:** Modelo estratégico que organiza las capacidades de una organización en servicios **autónomos, interoperables y reutilizables**.
- **Principios clave:** Abstracción (el servicio es una "caja negra"), bajo acoplamiento e interoperabilidad (independiente del lenguaje o plataforma).
- **Componentes:** Servicio (interfaz y contrato), Proveedor, Consumidor, Registro de Servicios y el **ESB (Bus de Servicio Empresarial)** que gestiona la comunicación.



## 7. Microservicios

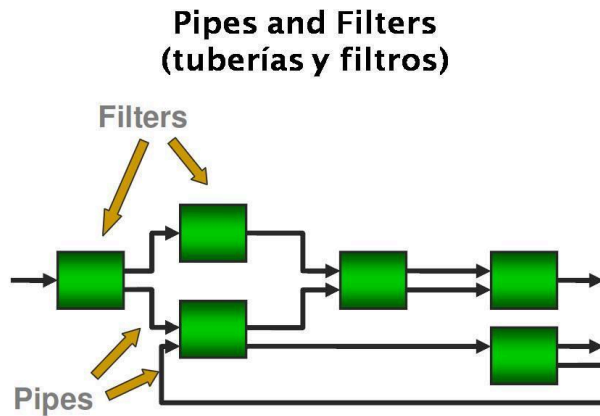
- **Concepto:** Evolución de la arquitectura monolítica que descompone una aplicación en servicios pequeños e independientes enfocados en una función de negocio específica.
- **Diferencias con Monolitos:** Los microservicios permiten el uso de diferentes tecnologías por servicio y ofrecen **escalabilidad horizontal selectiva** (solo se escala lo que tiene carga).
- **Fortalezas:** Agilidad en despliegue continuo y resiliencia (si uno falla, no cae todo).
- **Debilidad:** Alta complejidad operativa en monitoreo y consistencia de datos.



## 8. Tuberías y Filtros (Pipes and Filters)

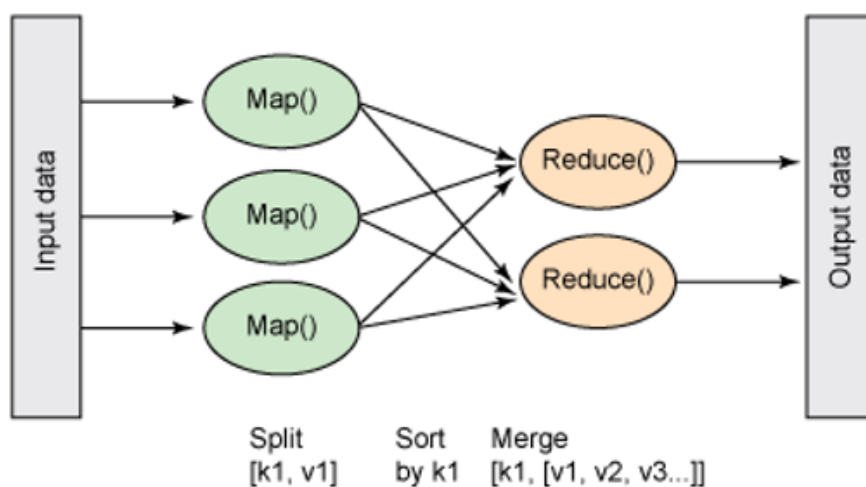
- **Concepto:** Divide un proceso complejo en etapas independientes llamadas **filtros**, que transforman datos y los pasan a la siguiente etapa a través de **tuberías**.
- **Características:** Bajo acoplamiento, permite el paralelismo y la reutilización de filtros en distintos procesos.
- **Debilidades:** Puede generar latencia y sobrecarga por copias de datos en cada etapa; es difícil identificar errores entre filtros.





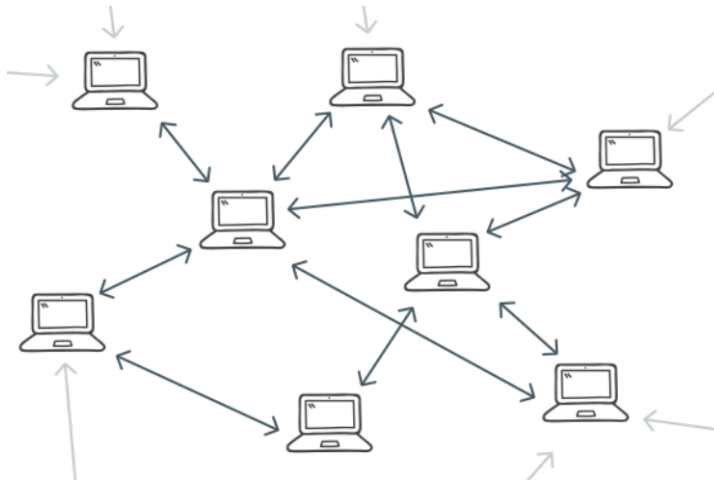
## 9. Map-Reduce

- **Concepto:** Modelo de procesamiento paralelo para datos a gran escala (Big Data).
- **Fases principales:**
  - **Map:** Convierte datos brutos en pares **clave-valor**.
  - **Shuffle & Sort:** Agrupa claves iguales para el mismo reductor.
  - **Reduce:** Procesa los grupos para producir un resultado consolidado.
- **Ventajas:** Alta tolerancia a fallos y funciona en hardware convencional (bajo costo).
- **Desventajas:** Alta latencia por dependencia de escritura en disco duro.



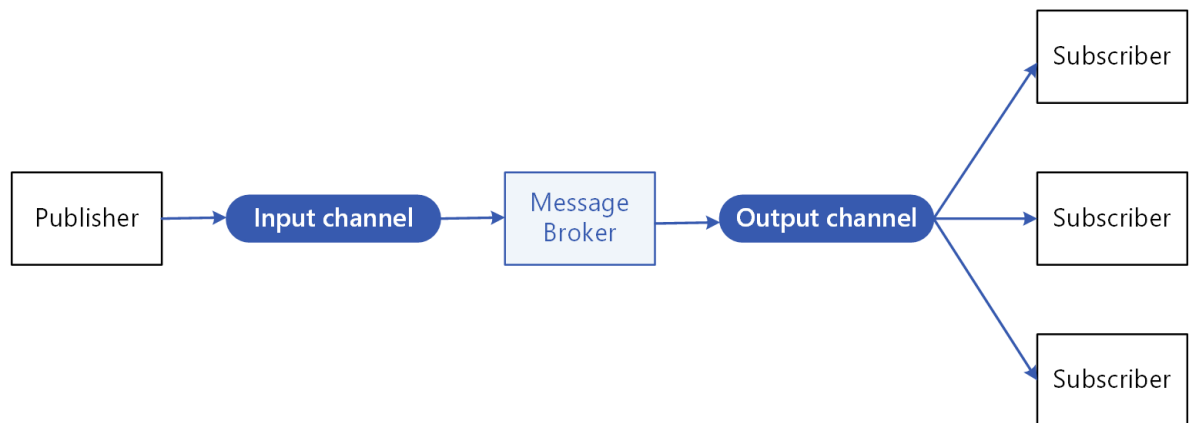
## 10. Peer-to-Peer (P2P)

- **Concepto:** Red descentralizada donde todos los nodos actúan como **cliente y servidor simultáneamente**.
- **Tipos:**
  - **Puro No Estructurado:** Usa inundación (Query Flooding) para buscar recursos entre vecinos conocidos.
  - **Puro Estructurado:** Usa tablas hash distribuidas (DHT) para localizar recursos mediante claves hash.
  - **Híbrido:** Usa un servidor central (**Tracker**) que solo mantiene un índice de quién tiene qué fragmento.



## 11. Publicador - Suscriptor (Pub/Sub)

- **Concepto:** Modelo de mensajería asíncrono donde los **Publicadores** envían mensajes a un canal lógico (**Topic**) sin conocer quién los recibirá.
- **Intermediario:** Un **Broker** recibe, almacena y distribuye los mensajes a los **Suscriptores** que tengan una suscripción activa a ese tema.
- **Ventaja principal:** Alto nivel de desacoplamiento; los servicios reaccionan a eventos sin depender directamente entre sí.



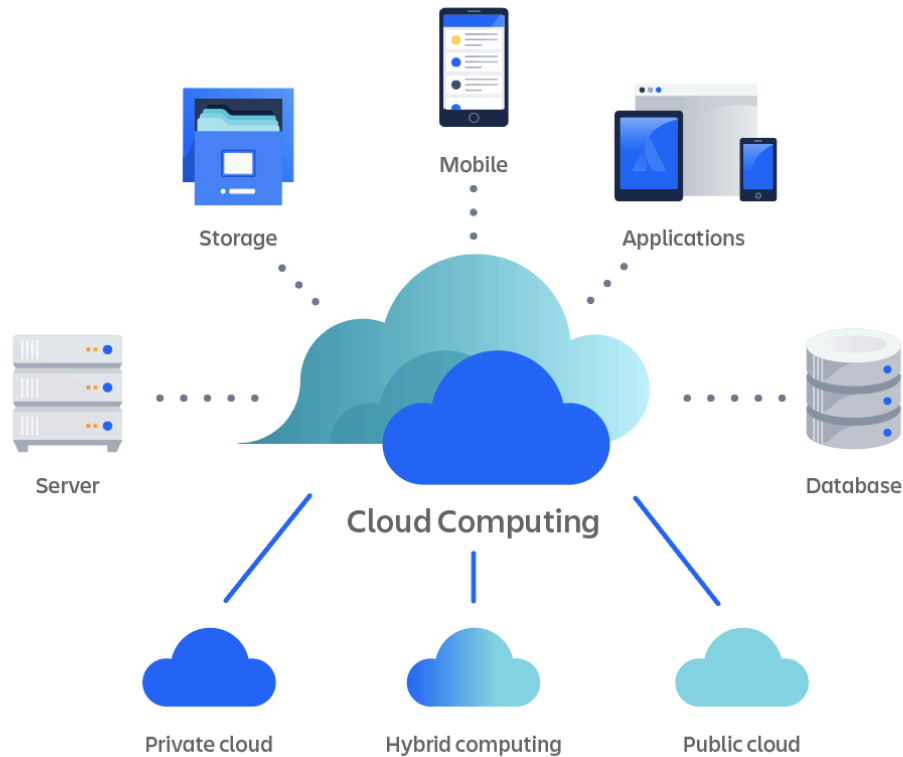
## 12. Arquitectura Orientada a Eventos

- **Concepto:** Los productores detectan cambios significativos (eventos) y los envían a un **Event Bus** (intermediario).
- **Funcionamiento:** Los consumidores se suscriben a tópicos y reaccionan a los eventos de forma asíncrona.
- **Dato Clave:** Es fundamental para sistemas que deben manejar alta demanda y tráfico distribuido globalmente.

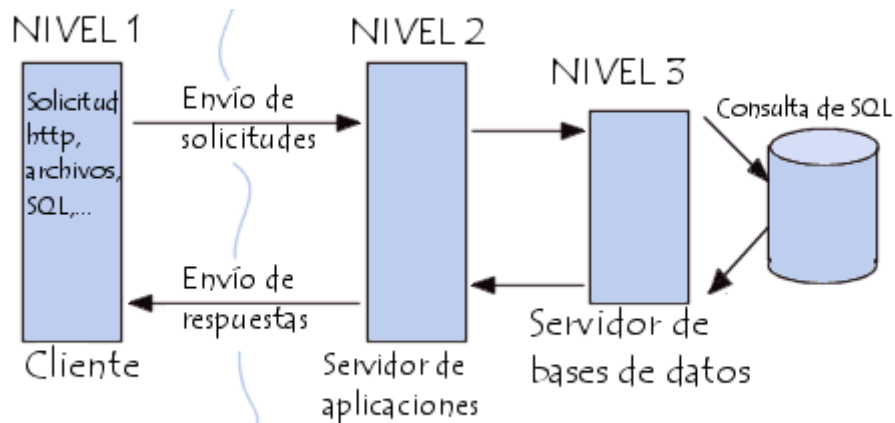


## 13. Cloud Computing y Arquitecturas Multinivel (N-Tier)

- **Cloud Computing:** No es una arquitectura en sí, sino un paradigma de entrega de servicios (IaaS, PaaS, SaaS, FaaS).



- **Arquitectura Multinivel (N-Tier):** Organiza el sistema en niveles separados física y lógicamente.
  - **3 Niveles:** Presentación, Aplicación (Lógica) y Datos.
  - **4 Niveles:** Suele separar el Servidor Web del Servidor de Aplicación para mayor granularidad.
- **Nota:** La diferencia clave es que una **capa** es una división lógica, mientras que un **nivel** implica separación física en diferentes servidores.



## **Tema 2: Diseño de Software. Diseño de Interfaz de Usuario.**

**Del Análisis al Diseño. Conceptos Principales del Diseño. Diseño de Interfaz de Usuario. Diseño de Componentes. Independencia Funcional. El procedimiento de derivación. Análisis de transacciones y transformación. Evaluación y refinamiento: criterios, acoplamiento y cohesión. Documentación del Diseño. Diseño Ux/ Ui. Técnicas y consideraciones.**

### **El Proceso de Diseño**

El diseño es un **proceso iterativo** donde se aplican técnicas y principios para definir un sistema con el detalle suficiente para su **realización física**, traduciendo los requisitos en una representación del software. Se divide en cuatro áreas principales:

- **Diseño de Datos:** Transforma el modelo de información en estructuras de datos (Modelo Relacional).
- **Diseño Arquitectónico:** Define la estructura modular del programa.
- **Diseño de Interfaz:** Establece la comunicación del software con otros sistemas y con los usuarios.
- **Diseño Procedimental:** Describe detalladamente los componentes del software.

### **Definición de la Arquitectura del Sistema**

En esta fase se definen las **particiones físicas**, la descomposición lógica en subsistemas, la ubicación de estos en las particiones y la infraestructura tecnológica necesaria.

### **Diseño de Interfaz de Usuario (UI) y Módulos**

El objetivo es diseñar para cada subsistema la **estructura modular** de sus procesos.

- Se realiza el diseño detallado de la interfaz de pantalla e impresa.
- **Regla fundamental:** La interfaz de usuario debe corresponderse con la estructura modular del sistema.

## Calidad del Diseño: Acoplamiento y Cohesión

Para garantizar la calidad, los módulos deben ser lo más **independientes** posible y ejecutar una **única función**. Se utilizan dos criterios principales:

- **Acoplamiento:** Clasifica el grado de independencia entre módulos. El objetivo es **minimizarlo** (bajo acoplamiento) eliminando relaciones innecesarias, reduciendo el número de conexiones y tratando a los módulos como "cajas negras".
- **Cohesión:** Mide la intensidad de la asociación funcional de los elementos dentro de un módulo. Se busca obtener módulos **altamente cohesivos**, donde todos sus elementos estén fuertemente relacionados entre sí para cumplir una tarea.

## Descomposición (Factoring)

Es la separación de funciones de un módulo en otros nuevos para mejorar el diseño. Se realiza por las siguientes razones:

- **Tamaño:** Un módulo ideal tiene entre **30 y 60 líneas** (media página a una página).
- **Claridad y Reutilización:** Evita la duplicación de código y crea módulos generales que pueden formar bibliotecas.
- **Separación de Trabajo y Administración:** Los módulos de nivel superior deben **coordinar y tomar decisiones** (gerencia), mientras que los de nivel inferior realizan el **trabajo** (cálculo, entrada/salida).

## Estrategias y Principios de Diseño

El diseño se basa en la **abstracción, modularidad, encapsulamiento y ocultamiento de información**. Existen dos estrategias principales para derivar la estructura modular:

- **Análisis de Transformaciones:** Se basa en el flujo de datos a través del sistema.
- **Análisis de Transacciones:** Se utiliza cuando un único elemento de datos provoca el inicio de uno de varios caminos de acción posibles.

### **Verificación, Pruebas y Documentación**

- **Verificación:** Garantiza la calidad técnica y la coherencia entre los modelos de diseño.
- **Plan de Pruebas:** Se definen a niveles unitarios, de integración, de implantación y de aceptación.
- **Documentación:** Es **vital** para que el diseño sea entendible por personas ajenas a su creación y sirva como especificación precisa para la etapa de codificación. Debe incluir la documentación de terminadores, almacenes, flujos y procesos.

**Diseño de Experiencia de Usuario: etapas, actividades, técnicas y herramientas.**

### **Concepto y Rol del Diseñador**

El Diseño de Experiencia de Usuario (UXD) es un enfoque integrador que une la **Arquitectura de Información (AI)** y el **Diseño de Interacción (IxD)** desde las primeras etapas de creación del software.

- **El Diseñador como Mediador:** El diseñador actúa como un puente entre las necesidades comunicativas del **cliente (emisor)** y las necesidades informativas y funcionales de los **usuarios (receptores)**.

- **Los Tres Círculos de Morville:** Todo proyecto de diseño es relativo y depende de la intersección de tres elementos: el **contexto**, el **contenido** y los **usuarios**.

## **Etapas del Proceso de Diseño de UX**

El proceso se puede abordar de forma lineal o, preferiblemente, **iterativa**, donde se repiten ciclos hasta lograr el producto deseado,. Las cuatro etapas principales son:

1. **Investigación:** Es la obtención de toda la información posible sobre el proyecto, los usuarios y el contexto. Se definen objetivos, se caracterizan perfiles de usuarios (receptores), se estudia el modelo de negocio (DAFO/FODA) y se analizan productos similares (benchmarking),.
2. **Organización:** En esta fase se estructuran y jerarquizan las temáticas y los contenidos en correspondencia con las necesidades identificadas. Se definen los flujos funcionales que tendrá el software.
3. **Diseño:** Se plasman los resultados en elementos tangibles como estructuras (taxonomías), diagramas de funcionamiento, pantallas (**wireframes**), etiquetado y **prototipos** de bajo y alto nivel.
4. **Prueba:** Se comprueba la calidad y robustez del diseño propuesto mediante pruebas con clientes y usuarios para asegurar que se resuelven las necesidades identificadas.

## **Técnicas de Trabajo**

Las técnicas no están ligadas obligatoriamente a una etapa y pueden mezclarse según el objetivo. Algunas de las más destacadas son:

- **Búsqueda de información:** Tormenta de ideas (brainstorming), grupos de enfoque (focus groups), entrevistas y creación de "Personas".
- **Organización: Card sorting** (organización de tarjetas) para jerarquizar contenidos y diagramas de afinidad,.



- **Funcionamiento y Diseño:** Análisis de tareas, flujos de procesos, diagramas de presentación (wireframes) y prototipado digital.
- **Evaluación:** Pruebas con usuarios, evaluación heurística y técnicas avanzadas como **EyeTracking** (seguimiento visual) o **MouseTracking**.

## Herramientas Recomendadas

- **Para encuestas:** Netquest o e-encuestas.
- **Para mapas y conceptos:** MindManager, CMapTools o Freemind.
- **Para diagramas y prototipos:** **Axure RP Pro**, Microsoft Visio, SmartDraw y OmniGraffle. Actualmente, las aplicaciones táctiles han empezado a sustituir el papel y lápiz tradicional en la creación de estos artefactos.

## Elementos de la UX (Modelo de Garrett)

El diseño se construye desde lo abstracto (concepción) hacia lo concreto (culminación) pasando por cinco niveles: **Estrategia** (necesidades del usuario), **Alcance** (especificaciones), **Estructura** (diseño de interacción), **Esqueleto** (diseño de interfaz) y **Superficie** (diseño visual).

### Tema 3: Modelo de Datos.

El modelamiento de datos. Modelo entidad relación de Chen. Modelo conceptual de datos. Modelo conceptual canónico. Refinamiento del modelo conceptual de datos. Modelo lógico de datos. Fases cualitativa, cuantitativa y de optimización.

### Conceptos y Características del Modelo de Datos

El modelo de datos se conforma mediante la definición de las perspectivas **lógica y física** del sistema. Un buen modelo no se logra de inmediato, sino que requiere de **refinamientos sucesivos** para alcanzar la calidad esperada.

- **Objetivo:** Almacenar información sin redundancia innecesaria y permitir un acceso fácil.
- **Factores Críticos:** Es vital trabajar de forma interactiva con los usuarios, seguir una metodología y considerar tanto la estructura como la **integridad de la información**.
- **Propiedades Deseables:** El modelo debe ser **flexible** (usar campos parametrizables), **mantenible** (cambios poco costosos) y **portable**.
- **Beneficios:** Permite elegir la tecnología óptima, facilita la comprensión del usuario antes de implementar el sistema y simplifica futuras migraciones de bases de datos.

### Modelo Entidad-Relación (Chen)

Es la herramienta para representar el modelo lógico. Sus componentes principales son:

- **Entidades:** Objetos del mundo real. Se dividen en **Regulares** (existencia propia) y **Débiles** (dependen de otra entidad).
- **Atributos:** Propiedades de las entidades o relaciones. Pueden ser **compuestos** (ej. una fecha dividida en día/mes/año) o identificadores (claves).

- **Relaciones:** Asociaciones entre entidades donde se puede indicar el **rol** que cada una desempeña.
- **Cardinalidad:** Indica el número de relaciones en las que una entidad puede aparecer. Se define por un **mínimo** (0 o 1) y un **máximo** (1 o muchos).

## Reglas de Negocio e Integridad

Para proporcionar integridad y consistencia, se deben definir reglas que afecten las operaciones básicas:

- **Reglas de Insertar:** Definen qué hacer cuando se añade una ocurrencia y no existe la entidad de la que depende (ej. no permitir un pedido si el proveedor no existe o crearlo en el momento).
- **Reglas de Borrar:** Definen qué ocurre con los datos conectados cuando se elimina una entidad (ej. borrar pedidos si se borra al proveedor o reasignarlos a uno por defecto).

## Clasificación y Flujo de los Modelos

El diseño de datos sigue una evolución desde la percepción humana hasta el almacenamiento físico:

1. **Mundo Real:** Información tal cual se percibe.
2. **Esquema Conceptual:** Independiente del Sistema Gestor de Bases de Datos (DBMS).
3. **Esquema Canónico:** Datos en formato cercano al ordenador, normalizados y sin redundancia.
4. **Esquema Interno:** Según el modelo específico del DBMS (ej. Oracle).
5. **Base de Datos Física:** Datos almacenados directamente en el disco.

## Independencia de Datos

El esquema conceptual debe ser independiente del físico y del lógico:

- **Independencia Física:** Cambios en el hardware o sistema operativo no deben afectar al esquema conceptual.
- **Independencia Lógica:** Cambios en el esquema conceptual no deben afectar la vista de las aplicaciones (esquemas externos).

## Normalización (Codd)

Es un proceso reversible para simplificar los datos y asegurar que cualquier requerimiento pueda ser satisfecho.

- **1FN (Primera Forma Normal):** No hay atributos repetitivos y los valores son atómicos.
- **2FN (Segunda Forma Normal):** Está en 1FN y los atributos dependen totalmente de la clave primaria.
- **3FN (Tercera Forma Normal):** Está en 2FN y los atributos dependen de forma no transitiva de la clave.
- **4FN y 5FN:** Se encargan de dependencias de valores múltiples y de producto.

## Modelo Multidimensional

Usado para análisis masivo de datos.

- **Hechos o Medidas:** Valores cuantitativos almacenados en el "cubo".
- **Dimensiones:** Ejes descriptivos (ej. Producto, Localización, Tiempo).
- **Esquemas:** **Estrella** (hecho central con dimensiones alrededor), **Copo de Nieve** (dimensiones normalizadas en jerarquías) y **Constelación** (colección de múltiples estrellas).

#### **Tema 4. Diseño Orientado a Objetos.**

**De los Requisitos al diseño. Objetos Y Clases. Proceso del Diseño Orientado a Objetos. Contexto del Sistema y Modelos de Utilización. Diseño de la Arquitectura. Identificación de Objetos. Modelo de Diseño. Especificación de la Interfaz de Objetos. La documentación del Diseño y la importancia de la trazabilidad. Objetos Concurrentes. Evolución del diseño. Diseño de Software de Tiempo Real. Diseño de Interfaces de Usuario.**

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

██████████

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

© 2006 The Authors

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]



[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]